WEEK 04

탐색적 데이터 분석과 시각화

EDA(Exploratory Data Analysis) · Data Visualization

목차

01 EDA가 뭘까 2
02 데이터 첫인상 확인하기 3
03 숫자로 요약하기 (기술통계) 4
04 변수 하나씩 보기 (단변량 분석) 5
05 변수끼리 관계 보기 (이변량·다변량 분석) 7
06 숫자만 믿으면 안 되는 이유 9
07 좋은 그래프의 원칙 10
08 어떤 그래프를 쓸까 + 색 고르기 12
09 이렇게 그리면 안 된다 13
10 파이썬으로 그리기 14
11 이미지 데이터는 어떻게 볼까 15
12 정리 · EDA 체크리스트 16

01 EDA가 뭘까

탐색적 데이터 분석(EDA)은 데이터를 모델에 넣거나 본격적으로 분석하기 전에, 요약 통계와 그래프로 데이터를 이리저리 뜯어보면서 생김새와 문제점을 파악하는 과정이다. 통계학자 존 튜키(John Tukey)가 1977년에 같은 제목의 책을 내면서 널리 퍼진 말이다.

쉽게 말하면 요리 전에 재료 상태부터 확인하는 단계다. 재료가 상했는지, 양은 충분한지, 어떤 요리에 어울리는지 모르고 바로 불부터 켜면 결과가 좋을 리 없다. 데이터도 똑같아서, 어떤 모델을 쓰든 데이터를 제대로 모르면 이상한 결과가 나와도 왜 그런지 알 수가 없다.

EDA로 알아내려는 것

- 데이터가 내가 생각한 모양이 맞는지 (열의 의미, 자료형, 단위, 값의 범위)
- 빈 값, 튀는 값, 중복, 오타 같은 문제가 어디에 얼마나 있는지
- 각 변수가 어떻게 분포하는지, 변수들끼리 어떤 관계가 있는지
- 그래서 전처리·특성 선택·모델은 어떻게 가져가야 할지 (방향 잡기)

탐색적 분석 vs 확증적 분석

	탐색적 분석 (EDA)	확증적 분석 (CDA)
목적	데이터에서 패턴과 가설을 **찾는다**	미리 세운 가설이 맞는지 **검증한다**
주로 쓰는 것	그래프, 요약 통계, 표	가설검정, 신뢰구간, p-value
태도	정해진 질문 없이 열린 마음으로 둘러봄	정해진 질문 하나에 답함

튜키가 비유한 걸 빌리면, EDA는 탐정이 단서를 모으는 일이고 확증적 분석은 법정에서 그 단서로 판결하는 일이다. 순서상 EDA가 먼저다.

EDA 흐름

1 **구조 확인** 크기, 자료형, 결측 → 2 **기술통계** 평균, 퍼짐, 모양 → 3 **단변량 분석** 변수 하나씩 → 4 **이변량·다변량** 변수끼리 관계 → 5 **정리·반영** 전처리, 모델링

> **메모**
> EDA와 전처리는 순서가 딱 정해진 게 아니라 왔다 갔다 한다. 보다가 이상한 걸 발견하면 고치고(전처리), 고친 데이터를 다시 본다(EDA). 3주차가 결측치·이상치를 **어떻게 처리할지**였다면, 이번 주는 그걸 **어떻게 찾아내고 이해할지**에 가깝다.

02 데이터 첫인상 확인하기

이 정리본에서는 예시로 seaborn에 들어 있는 **펭귄 데이터(penguins)**를 쓴다. 남극 세 섬에서 펭귄 344마리를 측정한 데이터로, 종·섬·성별(범주형)과 부리 길이·부리 깊이·날개 길이·몸무게(수치형)가 있다. 크기가 작고 빈 값도 조금 섞여 있어서 EDA 연습용으로 많이 쓰인다.

제일 먼저 돌려 보는 코드

```
import seaborn as sns
df = sns.load_dataset("penguins")

df.shape                      # (344, 7) → 344행, 7열
df.head()                     # 앞 5줄 미리보기
df.info()                     # 열 이름, 자료형, 빈 값 아닌 개수
df.describe()                 # 수치형 열 요약 통계
df.isna().sum()               # 열별 결측치 개수
df.duplicated().sum()         # 완전히 같은 행 개수
df["species"].value_counts()  # 범주별 개수
```

펭귄 데이터에 돌려 보면 이런 것들이 나온다.

- 344행 × 7열. 수치형 4개(부리 길이·깊이, 날개 길이, 몸무게), 범주형 3개(종, 섬, 성별)
- 수치형 4개 열에 결측이 2개씩 있는데, 확인해 보니 **같은 펭귄 2마리**가 측정값이 통째로 비어 있음
- 성별(sex)은 11개가 비어 있음. 중복 행은 0개

결측이 "몇 개"인지보다 "어디에, 어떤 식으로" 있는지가 더 중요하다. 여러 열이 같은 행에서 같이 비어 있다면 측정 자체를 못 한 경우일 가능성이 크다.

변수 유형 정리

큰 분류	세부 유형	뜻	예시
범주형	명목형	순서 없는 이름표	종, 섬, 성별, 혈액형
	순서형	순서는 있지만 간격은 의미 없음	만족도(상·중·하), 학년
수치형	이산형	셀 수 있는 값	형제 수, 방문 횟수
	연속형	재는 값, 소수점 가능	부리 길이, 몸무게, 키

유형을 먼저 나누는 이유는 유형에 따라 쓸 수 있는 통계와 그래프가 달라지기 때문이다. 수치형은 평균·히스토그램, 범주형은 개수·막대그래프를 쓴다.

> **주의**
> 숫자로 적혀 있다고 다 수치형은 아니다. 우편번호, 학번, "1=남자 2=여자" 같은 코드는 숫자 모양의 범주형이다. 이런 열의 평균은 아무 의미가 없으니 `info()` 에서 자료형만 보고 넘어가지 말고 열의 뜻을 꼭 확인한다.

03 숫자로 요약하기 (기술통계)

기술통계는 데이터를 몇 개의 숫자로 요약하는 것이다. 크게 세 가지를 본다. **어디쯤 모여 있나(중심), 얼마나 흩어져 있나(퍼짐), 어느 쪽으로 치우쳤나(모양).**

중심 — 평균, 중앙값, 최빈값

- **평균(mean)**: 다 더해서 개수로 나눈 값. 극단값 하나에 크게 끌려간다.
- **중앙값(median)**: 크기순으로 줄 세웠을 때 가운데 값. 극단값이 있어도 잘 안 변한다.
- **최빈값(mode)**: 가장 많이 나온 값. 범주형 데이터에도 쓸 수 있다.

> **예시**
>
> 직원 5명의 연봉이 3,000 / 3,200 / 3,500 / 3,800 / 30,000(만 원)이면 평균은 8,700, 중앙값은 3,500이다. 사장 한 명 때문에 평균이 확 뛴다. 뉴스의 "평균 연봉"이 체감과 다른 이유가 이거라서, 한쪽으로 치우친 데이터는 중앙값을 같이 봐야 한다.

퍼짐 — 범위, 분산·표준편차, 사분위수

- **범위(range)** = 최댓값 − 최솟값. 계산은 쉽지만 극단값 하나에 좌우된다.
- **분산·표준편차**: 값들이 평균에서 평균적으로 얼마나 떨어져 있는지. 표준편차는 분산에 루트를 씌워 원래 단위로 돌린 것이라 해석이 편하다.
- **사분위수**: 줄 세운 데이터를 4등분하는 지점. Q1(25%), Q2(50% = 중앙값), Q3(75%). **IQR = Q3 − Q1**은 가운데 50%가 퍼진 폭이고, 극단값에 강하다.

최솟값, Q1, 중앙값, Q3, 최댓값 다섯 개를 묶어서 **다섯 숫자 요약(five-number summary)**이라고 하는데, 이걸 그림으로 그린 게 박스플롯이다(4장).

모양 — 왜도와 첨도

- **왜도(skewness)**: 좌우로 얼마나 비대칭인지. 0이면 대칭, 양수면 오른쪽 꼬리가 길고, 음수면 왼쪽 꼬리가 길다.
- **첨도(kurtosis)**: 꼬리가 얼마나 두꺼운지, 즉 극단값이 얼마나 자주 나오는지. pandas의 `kurt()`는 정규분포를 0으로 맞춘 값을 준다. 양수면 정규분포보다 극단값이 잦은 편.

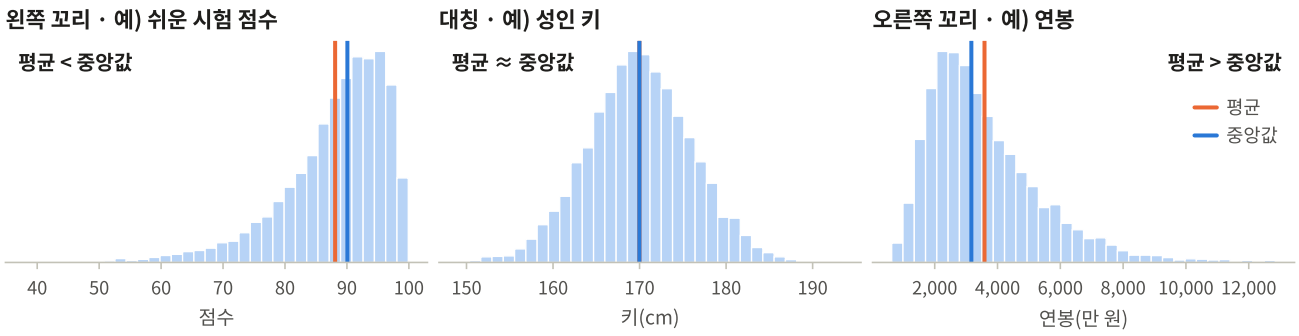


그림 1. 분포 모양에 따라 평균과 중앙값의 위치가 달라진다. 평균은 꼬리 쪽으로 끌려간다.

헷갈리기 쉬운 부분: "오른쪽으로 치우쳤다"는 말이 꼬리 방향을 말하는 건지 봉우리 위치를 말하는 건지 애매하다. 왜도는 **꼬리 방향** 기준이다. 연봉처럼 대부분은 왼쪽에 몰려 있고 소수가 오른쪽으로 길게 늘어진 게 양의 왜도다.

펭귄 데이터에 적용해 보면

	개수	평균	표준편차	최솟값	Q1	중앙값	Q3	최댓값	왜도
부리 길이(mm)	342	43.9	5.5	32.1	39.2	44.5	48.5	59.6	0.05
부리 깊이(mm)	342	17.2	2.0	13.1	15.6	17.3	18.7	21.5	−0.14
날개 길이(mm)	342	200.9	14.1	172	190	197	213	231	0.35
몸무게(g)	342	4,202	802	2,700	3,550	4,050	4,750	6,300	0.47

몸무게는 평균(4,202)이 중앙값(4,050)보다 크고 왜도도 양수라 오른쪽 꼬리가 조금 있다. 범위를 보면 음수 같은 말이 안 되는 값은 없다. 이렇게 `describe()` 한 번으로 "값이 상식적인 범위 안에 있나"를 먼저 걸러 볼 수 있다.

```
df.describe()                # 개수, 평균, 표준편차, 사분위수 한 번에
df["body_mass_g"].median()   # 4050.0
df.select_dtypes("number").skew()   # 왜도
df.select_dtypes("number").kurt()   # 첨도 (정규분포 = 0 기준)
```

04 변수 하나씩 보기 (단변량 분석)

단변량 분석은 변수 하나만 놓고 분포를 보는 것이다. 숫자 요약만으로는 분포 모양을 다 알 수 없어서 그래프를 같이 그린다.

수치형 ① 히스토그램

값의 범위를 여러 구간(bin)으로 나누고, 각 구간에 데이터가 몇 개 들어가는지를 막대로 그린 것이다. 막대 그래프처럼 생겼지만 x축이 연속된 숫자라서 막대끼리 붙어 있다.

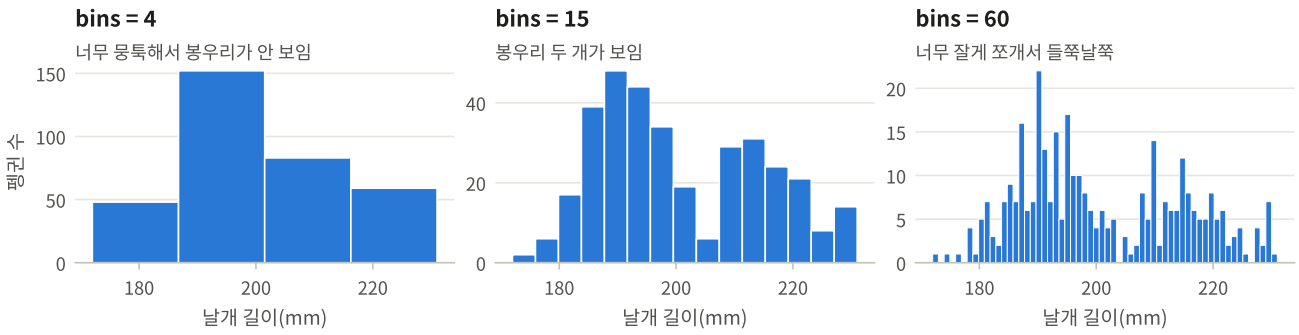


그림 2. 같은 데이터(펭귄 날개 길이)인데 구간 개수에 따라 전혀 다르게 보인다.

구간이 너무 적으면 중요한 모양이 뭉개지고, 너무 많으면 우연한 들쭉날쭉함까지 보인다. 정답 개수가 있는 게 아니라서 몇 가지로 바꿔 가며 그려 보는 게 좋다.

가운데 그림처럼 **봉우리가 두 개**면 성격이 다른 집단이 섞여 있다는 신호다. 실제로 펭귄 날개 길이는 젠투 펭귄이 따로 오른쪽 봉우리를 만든다. 이런 걸 발견하면 "종별로 나눠서 봐야겠다"는 다음 할 일이 생긴다.

수치형 ② KDE (밀도 곡선)

히스토그램을 매끄러운 곡선으로 바꾼 것(커널 밀도 추정)이다. 구간 경계에 덜 민감하고 여러 그룹을 겹쳐 그리기 좋다. 다만 곡선을 매끄럽게 이어 붙이다 보니 데이터가 없는 곳까지 꼬리가 나갈 수 있다. 예를 들어 음수가 나올 수 없는 값인데 곡선이 0 아래까지 그려지기도 하니 그대로 믿지는 말 것.

수치형 ③ 박스플롯

3장의 다섯 숫자 요약을 그림으로 그린 것이다. 상자의 아래·위가 Q1과 Q3, 가운데 선이 중앙값이다.

헷갈리기 쉬운 건 **수염**이다. 수염 끝은 최솟값·최댓값이 아니라, 상자에서 **1.5×IQR 안쪽에 있는 가장 먼 값**까지다. 그 밖에 있는 값은 점으로 따로 찍히고 이상치 후보로 본다.

장점은 여러 그룹을 나란히 놓고 비교하기 좋다는 것. 단점은 분포 모양이 숨는다는 것. 봉우리가 두 개여도 박스플롯에서는 안 보인다. 그래서 히스토그램과 같이 보는 게 좋다.

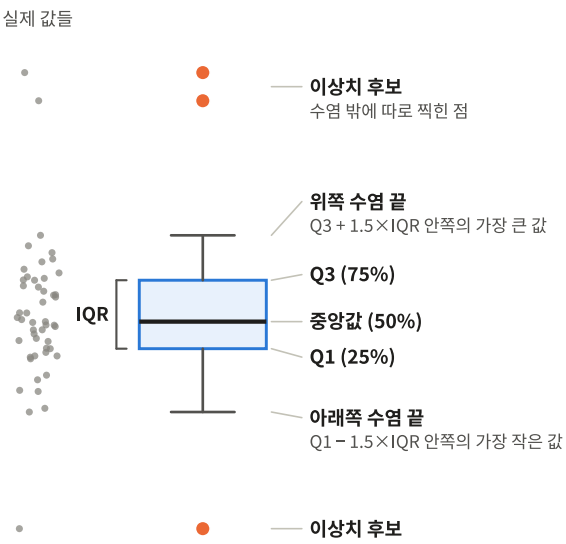


그림 3. 박스플롯 읽는 법

이상치 찾기

- **IQR 규칙**: Q1 − 1.5×IQR보다 작거나 Q3 + 1.5×IQR보다 크면 이상치 후보. 박스플롯의 점들이 바로 이것이다. 분포 모양을 가정하지 않아서 무난하게 쓴다.
- **Z-점수**: z = (값 − 평균) / 표준편차. 보통 $|z| > 3$이면 이상치 후보. 정규분포에 가까운 데이터에서 잘 맞는다.

```
q1, q3 = df["body_mass_g"].quantile([0.25, 0.75])
iqr = q3 - q1
mask = (df["body_mass_g"] < q1 - 1.5*iqr) | (df["body_mass_g"] > q3 + 1.5*iqr)
df[mask]        # 이상치 후보 행 보기 → 펭귄 데이터는 0개
```

주의

이상치 "후보"라고 쓴 이유가 있다. 기준을 넘었다고 바로 지우면 안 되고, 입력 실수인지(키 1750cm) 진짜 드문 값인지(아주 큰 펭귄) 먼저 확인한다. 진짜 값이면 오히려 중요한 정보일 수 있다. 처리 방법은 3주차 정리 참고.

범주형 — 빈도표와 막대그래프

범주형은 범주별 개수와 비율을 본다. 막대는 **많은 순서대로 정렬**하면 훨씬 읽기 편하고, 범주 이름이 길면 가로 막대가 낫다. 막대 색은 한 가지면 충분하다.

```
df["species"].value_counts()
df["species"].value_counts(normalize=True)  # 비율
```

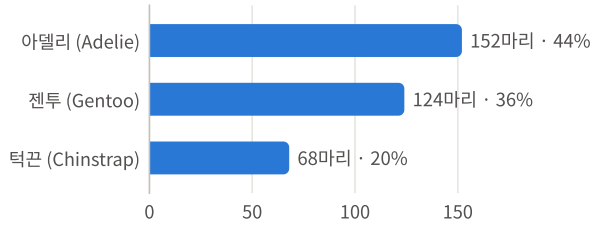


그림 4. 종별 마릿수

범주형에서는 거의 안 나오는 **희귀 범주**와, 같은 값인데 다르게 적힌 **표기 불일치**("Male", "male", "M")도 같이 확인한다. 그리고 예측할 정답(타깃) 변수는 꼭 따로 분포를 본다. 한 클래스가 90%를 넘는 식으로 불균형이 심하면 정확도만으로는 모델 성능을 판단할 수 없다.

05 변수끼리 관계 보기 (이변량·다변량 분석)

변수 두 개 이상을 같이 보는 단계다. 어떤 변수 조합이냐에 따라 쓰는 방법이 다르다.

조합	숫자로 보기	그래프로 보기
수치 × 수치	상관계수	산점도
범주 × 수치	그룹별 평균·중앙값 (`groupby`)	그룹별 박스플롯, 바이올린 플롯
범주 × 범주	교차표 (`pd.crosstab`)	묶은 막대, 누적 막대

수치 × 수치 — 산점도와 상관계수

산점도는 한 변수를 x축, 다른 변수를 y축에 놓고 점을 찍은 것이다. 이 점들이 얼마나 직선에 가깝게 모여 있는지를 숫자 하나로 나타낸 게 **상관계수 r**이다.

- r은 −1 ~ +1. 양수면 같이 커지고, 음수면 한쪽이 커질 때 다른 쪽은 작아진다.
- 절댓값이 클수록 직선에 가깝다. 대충 0.7 이상이면 강함, 0.3~0.7이면 보통, 0.3 미만이면 약함으로 보는데 분야마다 기준은 다르다.

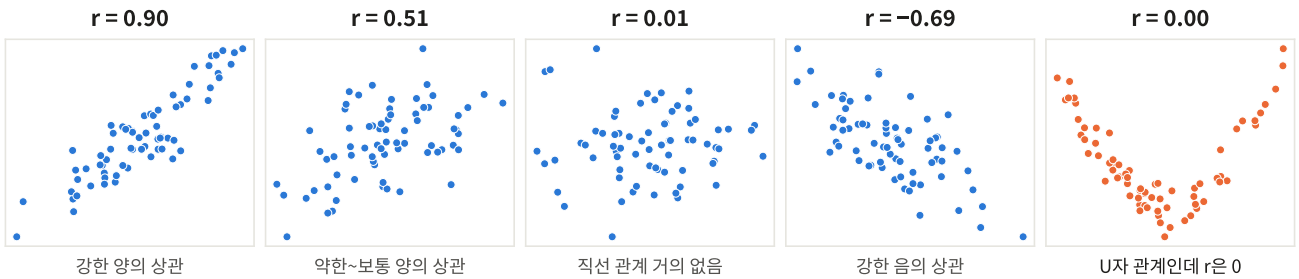


그림 5. 상관계수별 산점도. 맨 오른쪽은 관계가 뚜렷한데도 r이 0이다.

맨 오른쪽 그림이 중요하다. 상관계수는 **직선 관계만** 잡는다. U자처럼 휘어진 관계는 r이 0으로 나와서 "관계없음"으로 착각하기 쉽다. 그래서 r만 보지 말고 산점도를 꼭 같이 본다.

피어슨 vs 스피어만

보통 말하는 상관계수는 **피어슨** 상관계수로, 직선 관계를 재고 이상치에 민감하다. **스피어만** 상관계수는 값 대신 순위로 계산해서, 직선이 아니어도 한 방향으로 꾸준히 증가·감소하면 높게 나오고 이상치에도 덜 흔들린다.

`df.corr(method="spearman")`

범주 × 수치, 그리고 색으로 세 번째 변수 넣기

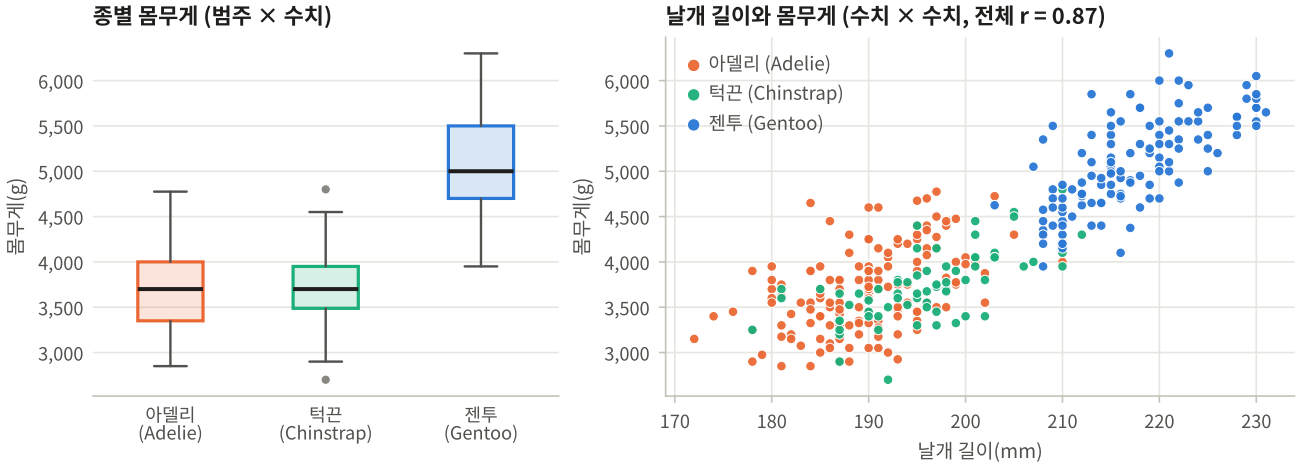


그림 6. 왼쪽: 종별로 나눈 몸무게. 오른쪽: 날개 길이와 몸무게의 산점도에 종을 색으로 표시

왼쪽 그림을 보면 젠투가 다른 두 종보다 확실히 무겁다. 오른쪽은 날개가 길수록 무거운 강한 양의 관계(r = 0.87)인데, 종을 색으로 칠하니 젠투가 오른쪽 위에 따로 무리를 이룬다는 것까지 보인다. 이렇게 색(hue), 크기, 그룹별 작은 그래프(facet)로 변수를 하나 더 얹을 수 있다.

범주 × 범주 — 교차표

```
pd.crosstab(df["species"], df["island"])

island     Biscoe  Dream  Torgersen
Adelie         44     56         52
Chinstrap       0     68          0
Gentoo        124      0          0
```

아델리는 세 섬 모두에 있는데, 턱끈은 Dream 섬에만, 젠투는 Biscoe 섬에만 있다. 이런 건 범주별 개수만 따로 봐서는 절대 모른다. 이 경우 "섬"이 사실상 종을 알려 주는 정보라서, 모델에 둘 다 넣을 때 이 점을 알고 있어야 한다.

여러 변수 한꺼번에 — 산점도 행렬과 상관 히트맵

산점도 행렬(pairplot)은 수치형 변수의 모든 쌍을 산점도로 한 번에 그린다. 변수가 4~6개 정도일 때 전체 그림을 잡기 좋다.

상관 히트맵은 상관계수 표를 색으로 칠한 것이다. 0을 가운데 회색으로 두고 양수·음수를 다른 색으로 칠하면 읽기 쉽다. 위아래가 대칭이라 절반만 그려도 된다.

날개 길이와 몸무게(0.87)처럼 서로 상관이 아주 높은 변수들은 비슷한 정보를 담고 있다는 뜻이다. 선형 모델에서는 이런 변수끼리 **다중공선성** 문제가 생길 수 있어서 특성 선택할 때 참고한다.

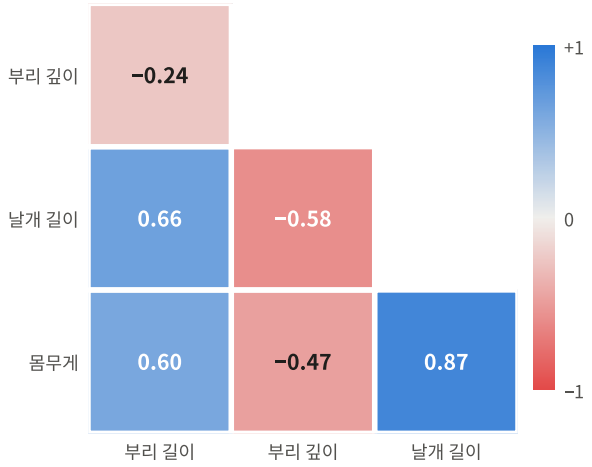


그림 7. 펭귄 수치형 변수의 상관 히트맵

```
sns.pairplot(df, hue="species")                    # 산점도 행렬
corr = df.select_dtypes("number").corr()
sns.heatmap(corr, annot=True, cmap="RdBu", vmin=-1, vmax=1)   # 히트맵
```

06 숫자만 믿으면 안 되는 이유

앤스컴 콰르텟 (Anscombe's quartet)

통계학자 앤스컴이 1973년에 만든 데이터 4세트다. 네 세트 모두 요약 통계가 거의 똑같다.

x 평균	y 평균	x 분산	y 분산	상관계수	회귀선
9	7.50	11	4.12	0.82	y = 3 + 0.5x

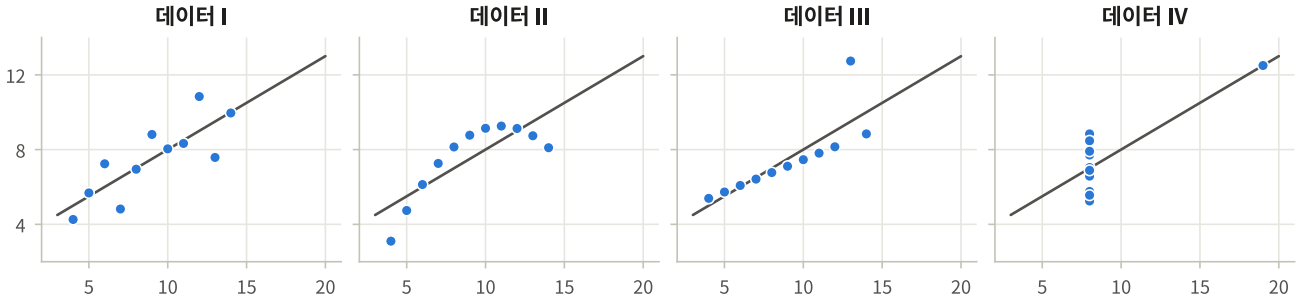


그림 8. 앤스컴 콰르텟. 회귀선은 넷 다 같다.

그런데 그려 보면 완전히 다르다. Ⅰ은 평범한 직선 관계, Ⅱ는 곡선, Ⅲ은 직선인데 튀는 점 하나가 기울기를 끌어올렸고, Ⅳ는 x가 거의 다 같은데 점 하나 때문에 상관이 생겼다. **요약 통계는 꼭 그래프로 확인해야 한다**는 걸 보여 주는 대표 예시다. (2017년에는 같은 통계를 가지면서 공룡 모양이 되는 "Datasaurus" 데이터도 나왔다.)

상관관계 ≠ 인과관계

아이스크림 판매량과 물놀이 사고 건수를 그려 보면 같이 올라간다. 그렇다고 아이스크림이 사고를 일으키는 건 아니다. 둘 다 **더운 날씨** 때문에 늘어난 것이다. 이렇게 두 변수에 같이 영향을 주는 제3의 변수를 교란 변수라고 한다. EDA에서 찾은 관계는 "여기 뭔가 있다"는 힌트까지만이고, 원인이라고 말하려면 실험 같은 다른 근거가 필요하다.

심슨의 역설

전체로 보면 한 방향인 관계가, 그룹으로 나누면 반대 방향이 되는 현상이다. 펭귄 데이터에도 있다.

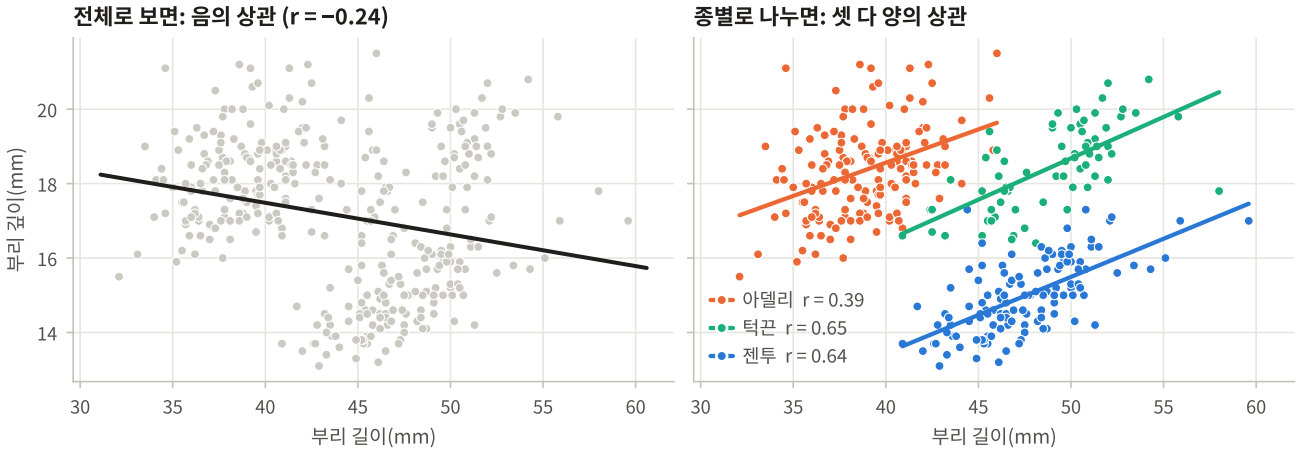


그림 9. 부리 길이와 부리 깊이. 전체 상관은 음수인데 종별로는 모두 양수다.

종을 무시하고 보면 "부리가 길수록 얇다"는 결론이 나온다. 하지만 종별로 나누면 세 종 모두 "부리가 길수록 두껍다". 젠투가 부리는 길면서 얇은 종이라서 전체 그림이 뒤집혀 보인 것이다. 그래서 EDA를 할 때는 **종, 성별, 지역 같은 그룹 변수로 나눠서도** 한 번씩 그려 봐야 한다.

07 좋은 그래프의 원칙

그래프를 그리는 두 가지 목적

	탐색용 (나 보려고)	설명용 (남에게 보여 주려고)
목표	데이터에서 뭔가를 발견하기	발견한 것 하나를 정확히 전달하기
방식	빠르게, 많이, 대충 그려 봄	메시지 하나를 정하고 그게 한눈에 보이게 다듬음
예	EDA 중에 그리는 히스토그램 수십 장	발표 자료, 보고서에 들어가는 그래프

EDA 단계에서는 예쁘게 그리느라 시간 쓸 필요가 없다. 대신 발표나 보고서에 넣을 그래프는 아래 원칙들을 신경 쓴다.

원칙 1. 데이터 잉크 비율을 높인다

에드워드 터프티(Edward Tufte)가 말한 개념이다. 그래프에 쓰인 잉크 중 실제 데이터를 나타내는 잉크의 비율을 높이라는 것. 3D 효과, 그림자, 진한 격자선, 배경 그림처럼 정보는 없고 눈만 어지럽히는 장식을 **차트정크(chartjunk)**라고 하는데, 이런 걸 빼는 것만으로 그래프가 훨씬 깔끔해진다.

원칙 2. 사람 눈이 정확하게 읽는 방식을 쓴다

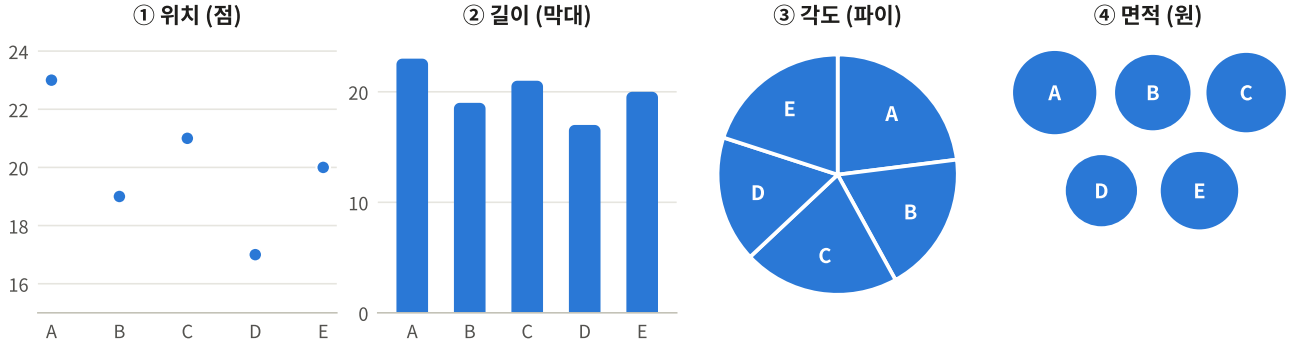


그림 10. 같은 숫자 다섯 개(A 23, B 19, C 21, D 17, E 20)를 네 가지 방식으로 그린 것. B와 E 중 뭐가 더 큰지 각각 비교해 보자.

①, ②에서는 E가 B보다 크다는 게 바로 보이는데 ③, ④에서는 거의 구분이 안 된다. 클리블랜드와 맥길(Cleveland & McGill)의 1984년 실험에 따르면 사람은 대체로 이 순서로 정확하게 읽는다.

같은 축 위의 위치 › **길이** › **각도·기울기** › **면적** › **부피·색의 진하기**

그래서 정확한 비교가 중요하면 막대나 점을 쓰고, 파이(각도)나 버블(면적)은 대략적인 느낌만 줄 때 쓴다.

원칙 3. 강조는 하나만

사람은 색, 크기, 위치 같은 차이는 의식하기도 전에 0.몇 초 만에 알아챈다(전주의적 속성). 그래서 하고 싶은 말과 관련된 것 하나에만 색을 주고 나머지는 회색으로 두면, 보는 사람이 어디를 봐야 할지 바로 안다.

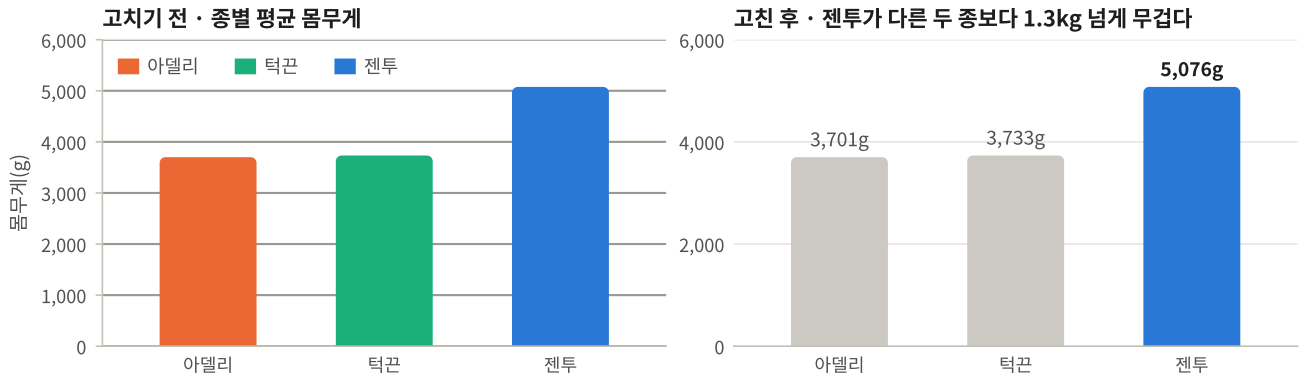


그림 11. 같은 데이터, 다른 그래프. 오른쪽은 강조 1개 + 결론을 적은 제목 + 막대 위 값 표시

왼쪽은 색이 셋인데 아무 의미가 없고, 범례는 x축 이름을 한 번 더 말할 뿐이며, 진한 격자선이 막대보다 눈에 띈다. 오른쪽은 제목에 결론을 적었다. 설명용 그래프라면 "종별 평균 몸무게"처럼 주제만 적는 것보다 **"젠투가 1.3kg 넘게 무겁다"처럼 결론을 적는 제목**이 훨씬 잘 전달된다.

원칙 4. 기본은 빼먹지 않는다

- 제목, 축 이름, **단위**(g인지 kg인지)를 쓴다.
- 범례를 찾아 색을 맞추게 하는 것보다, 가능하면 선 끝이나 막대 옆에 이름을 직접 쓴다.
- 숫자를 모든 점에 다 쓰지 말고 중요한 것에만 쓴다.
- 축을 자르거나 비율을 왜곡하지 않는다 (9장).

08 어떤 그래프를 쓸까 + 색 고르기

그래프는 "무엇을 보여 주고 싶은가"로 고른다. 자주 쓰는 경우를 표로 정리하면 이렇다.

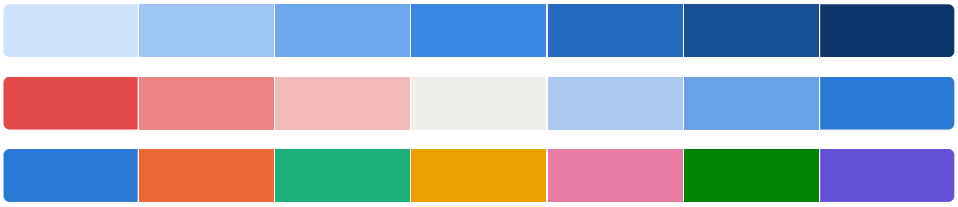
보여 주고 싶은 것	추천 그래프	메모
크기 비교	막대그래프, 점 그래프	범주가 많거나 이름이 길면 가로 막대 + 정렬
분포	히스토그램, KDE, 박스플롯, 바이올린	그룹 여러 개를 비교할 땐 박스플롯이 깔끔
시간에 따른 변화	선 그래프	x축은 시간 순서대로, 간격을 일정하게
두 수치의 관계	산점도	점이 많으면 투명도(alpha)를 낮추거나 hexbin
전체 중 비율(구성)	누적 막대, 파이	파이는 조각 5개 이하, 비율 차이가 클 때만
여러 변수 한눈에	산점도 행렬, 상관 히트맵	변수가 너무 많으면 관심 있는 것만 골라서
범주 × 범주 개수	히트맵, 묶은 막대	교차표를 색으로 칠하면 히트맵

색은 역할에 맞게

순차형
양의 크기 (인구, 매출)

발산형
가운데 0 기준 ± (상관계수, 증감률)

범주형
종류 구분 (종, 지역)



- **순차형**: 한 가지 색을 연한 것부터 진한 것까지. 값이 클수록 진하게.
- **발산형**: 가운데(0, 평균)는 회색 같은 중립색, 양쪽 끝은 서로 반대 느낌의 두 색. 그림 7의 상관 히트맵이 이 방식이다.
- **범주형**: 서로 잘 구분되는 색. 순서가 없는 범주에 진하기 차이를 주면 크고 작음이 있는 것처럼 오해한다. 많아야 6~8개, 그 이상이면 상위 몇 개 + "기타"로 묶는다.

> **색각 이상 고려하기**
> 남성의 약 5~8%는 빨강·초록 구분이 어렵다. 빨강과 초록만으로 좋음/나쁨을 나타내지 말고 모양, 라벨, 위치를 같이 쓴다. 연속값에는 matplotlib 기본 컬러맵인 **viridis**가 무난하다. 색각 이상이 있어도, 흑백으로 인쇄해도 밝기 순서가 유지되게 만든 컬러맵이다. 반대로 무지개색(jet)은 밝기가 들쭉날쭉해서 없는 경계가 생겨 보이니 피한다.

09 이렇게 그리면 안 된다

막대그래프의 y축 자르기

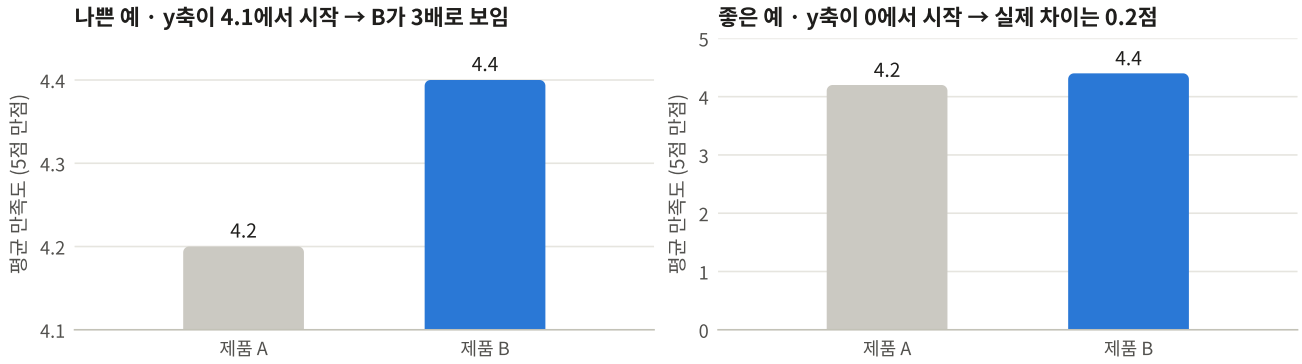


그림 12. 같은 데이터(4.2점 vs 4.4점)인데 y축 시작점만 다르다.

막대그래프는 **막대 길이**로 값을 비교하는 그래프라서 y축이 0에서 시작해야 한다. 왼쪽처럼 4.1부터 시작하면 0.2점 차이가 세 배 차이처럼 보인다. 광고나 뉴스 그래프에서 자주 보이는 수법이다. 반면 선 그래프는 변화 추세를 보는 게 목적이라 0에서 시작하지 않아도 괜찮은 경우가 많다.

그 밖에 자주 하는 실수

실수	왜 문제인가	대신
3D 그래프, 3D 파이	앞쪽 조각이 커 보이는 등 원근 때문에 크기가 왜곡된다	평면 막대그래프
y축 2개 (이중 축)	두 축 눈금을 마음대로 맞출 수 있어서 없는 관계도 있어 보이게 만들 수 있다	그래프를 둘로 나누기
비슷한 값의 파이 차트	각도는 비교가 어렵다 (그림 10)	막대그래프
색 10개 이상	구분도 안 되고 범례 찾느라 지친다	상위 몇 개 + 기타, 또는 그룹별 작은 그래프
수천 개 점이 겹친 산점도	점이 겹쳐서 어디가 빽빽한지 모른다	투명도 낮추기, hexbin, 2D 밀도 그림
순서 없는 막대 정렬	비교하려면 눈이 왔다 갔다 해야 한다	크기순 정렬 (단, 시간·순서형은 원래 순서)
축 이름·단위 없음	숫자가 무슨 뜻인지 모른다	축 이름에 단위까지

정리하면 다 같은 이야기다. 보는 사람이 **데이터를 있는 그대로, 빨리** 읽을 수 있게 하는 게 목적이고, 그걸 방해하는 건 다 빼는 게 맞다.

10 파이썬으로 그리기

라이브러리	특징	주로 쓸 때
matplotlib	가장 기본. 세세한 부분까지 다 조절할 수 있지만 코드가 길어진다	그래프를 하나하나 다듬을 때
seaborn	matplotlib 위에서 동작. 통계 그래프를 한 줄로 그리고 DataFrame과 잘 맞는다	EDA 할 때 빠르게
plotly	마우스로 확대하고 값을 볼 수 있는 인터랙티브 그래프	웹, 대시보드로 보여 줄 때

matplotlib의 구조: Figure와 Axes

Figure는 도화지 전체, **Axes**는 그 안에 들어가는 그래프 하나하나다. `plt.subplots(1, 3)` 이면 도화지 하나에 그래프 칸 3개를 만든다는 뜻. seaborn 함수에 `ax=` 를 넘겨 주면 원하는 칸에 그릴 수 있다.

```
import matplotlib.pyplot as plt
import seaborn as sns

df = sns.load_dataset("penguins")
fig, axes = plt.subplots(1, 3, figsize=(15, 4))   # 도화지 1장, 칸 3개

sns.histplot(data=df, x="flipper_length_mm", bins=15, ax=axes[0])
sns.boxplot(data=df, x="species", y="body_mass_g", ax=axes[1])
sns.scatterplot(data=df, x="flipper_length_mm", y="body_mass_g",
                hue="species", ax=axes[2])          # hue = 색으로 구분

axes[0].set_title("날개 길이 분포")
axes[1].set_ylabel("몸무게(g)")
plt.tight_layout()                                  # 칸끼리 겹치지 않게
plt.show()
```

EDA에서 자주 쓰는 seaborn 함수

함수	용도	함수	용도
`histplot`	히스토그램	`scatterplot`	산점도
`kdeplot`	밀도 곡선	`lineplot`	선 그래프 (추세)
`boxplot` / `violinplot`	그룹별 분포 비교	`heatmap`	상관 히트맵, 교차표
`countplot`	범주별 개수 막대	`pairplot`	산점도 행렬

한글이 네모로 깨질 때

matplotlib 기본 폰트에는 한글이 없어서 제목이나 축 이름에 한글을 쓰면 □□□로 나온다. 한글 폰트를 지정하면 된다.

```
plt.rcParams["font.family"] = "Malgun Gothic"   # 윈도우 (맥은 "AppleGothic")
plt.rcParams["axes.unicode_minus"] = False       # 마이너스 기호 깨짐 방지

# 코랩에서는 패키지 하나로 해결
# !pip install koreanize-matplotlib
import koreanize_matplotlib
```

> **메모**
>
> `ydata-profiling` 같은 자동 EDA 도구를 쓰면 한 줄로 통계와 그래프가 잔뜩 들어간 리포트가 나온다. 처음에 전체 그림을 잡기엔 편하지만, 결국 중요한 건 그래프를 보고 "왜 이렇지?"를 생각하는 부분이라 직접 그려 보는 연습은 따로 해 두는 게 좋다.

11 이미지 데이터는 어떻게 볼까

지금까지는 표 형태(정형) 데이터 이야기였다. 이미지 같은 비정형 데이터는 픽셀 하나하나의 평균이나 상관을 보는 게 큰 의미가 없어서 보는 포인트가 조금 다르다.

확인할 것	왜	방법
클래스별 이미지 수	불균형이 심하면 많은 쪽만 맞히는 모델이 됨	막대그래프
이미지 크기·가로세로 비율	리사이즈 크기를 정하고, 비율이 너무 다른 건 찌그러짐	가로×세로 산점도, 히스토그램
샘플 직접 보기	라벨이 틀린 이미지, 엉뚱한 이미지, 워터마크 등은 숫자로는 안 보임	클래스별로 몇 장씩 격자로 띄우기
밝기·색 분포	너무 어둡거나 밝은 이미지 확인, 정규화에 쓸 채널별 평균·표준편차	픽셀값 히스토그램
파일 상태	깨진 파일, 중복 이미지, 흑백·RGBA가 섞여 있으면 학습 중 에러	열어 보기, 채널 수 세기
(탐지 데이터) 박스	물체가 아주 작거나 한쪽에 몰려 있는지	박스 크기·위치 분포, 이미지당 물체 수

```
from pathlib import Path
from PIL import Image
import pandas as pd

rows = []
for p in Path("data/train").glob("*/*.jpg"):      # 폴더 이름 = 클래스
    with Image.open(p) as im:
        rows.append({"label": p.parent.name, "w": im.width,
                     "h": im.height, "mode": im.mode})
info = pd.DataFrame(rows)

info["label"].value_counts()     # 클래스별 개수 (불균형 확인)
info[["w", "h"]].describe()      # 크기 분포
info["mode"].value_counts()      # RGB / L(흑백) / RGBA 섞였는지
```

결국 표 데이터와 마찬가지로 "개수 → 분포 → 직접 보기" 순서다. 특히 이미지는 **직접 눈으로 보는 단계**를 건너뛰면 안 된다. 라벨이 잘못 붙은 이미지 몇 장이 성능을 생각보다 많이 깎아 먹는다.

12 정리 · EDA 체크리스트

새 데이터를 받았을 때 이 순서대로 보면 큰 건 놓치지 않는다.

구조

- ☐ 크기, 열의 뜻, 자료형 확인. 숫자 모양인데 사실 범주형인 열은 없는지
- ☐ 결측치 개수와 위치. 같은 행에서 같이 비어 있는지
- ☐ 중복 행, 표기 불일치("Male"/"male")

변수 하나씩

- ☐ `describe()`로 범위 확인. 음수 나이 같은 말이 안 되는 값은 없는지
- ☐ 히스토그램·박스플롯으로 치우침, 봉우리 개수, 이상치 후보 확인 (구간 수 바꿔 가며)
- ☐ 범주형 빈도와 희귀 범주. 타깃 변수의 불균형

관계

- ☐ 산점도와 상관계수를 같이 보기 (r만 믿지 않기)
- ☐ 그룹별 박스플롯, 교차표, 상관 히트맵
- ☐ 그룹 변수로 나눠서도 보기 (심슨의 역설). 상관을 인과로 착각하지 않기

마무리

- ☐ 발견한 점을 적어 두고 전처리·특성 선택·모델 선택에 반영
- ☐ 남에게 보여 줄 그래프는 메시지 하나 + 강조 하나 + 결론 제목 + 정직한 축

용어 한 번 더

EDA	모델링 전에 데이터를 요약·시각화하며 특징과 문제를 찾는 과정
다섯 숫자 요약	최솟값, Q1, 중앙값, Q3, 최댓값. 박스플롯의 재료
IQR	Q3 − Q1. 가운데 50%의 폭. 1.5×IQR 규칙으로 이상치 후보를 찾음
왜도 / 첨도	비대칭 정도(꼬리 방향) / 꼬리 두께(극단값이 잦은 정도)
상관계수 r	두 수치 변수의 직선 관계 세기, −1 ~ +1. 곡선 관계는 못 잡음
심슨의 역설	전체와 그룹별로 관계 방향이 반대로 나오는 현상
데이터 잉크 비율	그래프 잉크 중 데이터를 나타내는 잉크의 비율. 높을수록 좋음
차트정크	정보 없이 눈만 어지럽히는 장식 (3D, 그림자, 진한 격자)

한 줄로 줄이면, **숫자로 감을 잡고, 그래프로 확인하고, 이상한 건 일단 의심해 본다.** 다음 주(5주차)에 베이스라인 모델을 만들 때 여기서 본 것들이 전처리와 특성 선택의 근거가 된다.